· FORGE FDE ACADEMY · SPRINT 0 · DAY 4

# Problem Framing & Use Case Definition

Tonight we write down what DecisionStream AI actually is — in words a client, a regulator and an engineer would all accept.

2 hours | You build, I facilitate | Two artefacts leave this room

TONIGHT

# Two hours, and you do most of the talking

| | | |
|---|---|---|
| 0:00 – 0:15 | **The bad problem statement** | You find what is wrong with it |
| 0:15 – 0:45 | **The method** | Canvas · user stories · API contract · success criteria · NFRs |
| 0:45 – 1:30 | **The work** | Three lanes, working in parallel on your own industry |
| 1:30 – 1:50 | **The defence** | Each lane presents. One challenge question each. |
| 1:50 – 2:00 | **Commercial close** | Which of your accounts does this fit? |

**TWO THINGS LEAVE THIS ROOM TONIGHT** A completed Problem Framing Canvas for your pod's chosen industry, and version 0.1 of the API contract for POST /case-application. Both go into the Architecture Decision Record on Wednesday. Neither of them requires you to write code.

START HERE

# Read this problem statement

*“The client wants to use AI to make their case processing faster and more efficient using the latest LLM technology.”*

This went into a real proposal. Three people reviewed it. It was approved.

DISCUSS · 4 MIN

**In pairs. Write down everything that is wrong with this sentence. Be specific.**

*We will take answers from the room before I say anything.*

NOW THE HARDER QUESTION

# How many of those mistakes have you signed off on a proposal in the last two years?

Nobody is writing an answer down. But be honest with yourself, because that is the point of tonight.

WHAT WAS WRONG

# Six faults in one sentence

### It names a solution, not a problem

“AI” and “LLM” are answers. We have not yet said what question they answer.

### “Faster” has no starting number

Faster than what? If you do not know today's number, you cannot prove improvement.

### Nobody suffers in this sentence

No person appears. No role, no queue, no customer waiting. Problems happen to people.

### There is no constraint

Every real client has rules that cannot be broken. None are mentioned here.

### “Latest technology” is a preference

It is what the proposal writer wants, not what the client needs.

### You cannot fail it

And if you cannot fail it, you cannot succeed at it either. There is nothing to test.

**THE TEST** A problem statement you cannot fail is a wish. Framing is the work of turning a wish into something testable.

BEFORE WE START

# Words we will use tonight — no one needs to pretend

| Word | What it means, plainly | Why you should care |
|---|---|---|
| **Problem framing** | Writing down the real problem before choosing any solution | It decides whether the project can be judged at all |
| **User story** | One sentence: who wants what, and why | It keeps the client's voice in the build |
| **API** | How one computer system asks another to do something | It is the join between DecisionStream and everything else |
| **API contract** | A written promise: send me this, I return that | Teams build against it before the code exists |
| **OpenAPI** | The standard file format for writing that promise down | It is a document, not code. You can read it. |
| **POC** | Proof of concept — a build that proves one thing is possible | It does not have to be production-ready |
| **NFR** | Non-functional requirement — how well the system must work | This is where SLAs and penalties come from |
| **Latency** | How long a user waits for an answer | Usually the first number a client asks for |
| **Explainability** | Being able to say why the system gave this answer | Regulators ask this. So do customers. |

**GROUND RULE** If I use a word tonight that is not on this list and you are not sure of it, stop me. That is not a gap in your knowledge — it is a gap in my explanation.

PART 1

# The Problem Framing Canvas

Eight boxes. If you cannot fill them, you are not ready to build.



PART 1

# Why we use a canvas and not a document

WHAT

## One page, eight boxes

- Everyone sees the whole picture
- Gaps are visible, not buried
- It fits on a wall

WHY

## A document hides the holes

- Twenty pages can miss the main point
- A blank box cannot be missed
- It forces agreement, not wording

HOW

## Fill it with the client in the room

- Not afterwards, from notes
- Argue in front of them
- Leave with it signed

WHEN

## Before any architecture

- Before service selection
- Before estimates
- Before you promise a date

**FOR YOUR OWN ACCOUNTS** Many of you already do this in a discovery workshop. The canvas is not a new idea. What is new is treating it as a delivery artefact that goes into the architecture record — not a workshop output that is typed up and forgotten.

**The one-line test:** if a stranger reads your canvas and cannot tell you what the client should stop doing on Monday, it is not finished.

PART 1

# The eight boxes

1

## Who suffers?

Name the role. Not “the business”.

2

## What happens today?

The current path, step by step, with a number.

3

## What does it cost?

Hours, rupees, risk, delay. Pick a unit.

4

## Why has nobody fixed it?

The most important box on the page.

5

## What must not change?

Regulation, systems, people, contracts.

6

## What does better look like?

Describe the new Monday morning.

7

## How will we know?

The measure, and today's baseline for it.

8

## What are we NOT solving?

Written down, agreed, and shown to the client.

**BOXES 4 AND 8 SEPARATE SENIORS FROM JUNIORS** Box 4 tells you what will block you. Box 8 is what stops the client expecting five things when you promised one.

PART 1 · WORKED EXAMPLE

# The same canvas, filled in — credit underwriting

| Box | Filled in |
|---|---|
| **1 · Who suffers** | The credit officer, and the customer waiting for an answer |
| **2 · Today** | Officer opens 11 documents, finds figures by hand, cross-checks them, then decides. Six hours per case. |
| **3 · Cost** | Four of those six hours are clerical. Cases wait nine days in queue. We pay judgement rates for clerical work. |
| **4 · Why not fixed** | Documents arrive in different formats; the last automation attempt could not show where numbers came from, so risk refused it |
| **5 · Must not change** | A human signs every decision. Personal data stays in region. Every figure traceable to a page. |
| **6 · Better looks like** | Officer opens one screen with figures already extracted, each showing its source page. Judgement takes two hours. |
| **7 · How we know** | Officer hours per case: today six, target two point five. Queue days: today nine, target four. |
| **8 · NOT solving** | We are not automating the decision. We are not touching the customer notification. We are not re-platforming the case system. |

**THIS IS ONE OF FIVE** The same eight boxes work for insurance claims, citizen service requests, healthcare prior-authorisation and internal HR or vendor cases. Your pod picks tonight.

PART 1 · SCENARIO

# Talk about this

DISCUSS · 6 MIN

**Same client. Same situation. Two ways of writing it down. Which one can you price, and which one can you fail?**

1. Framing A — “Case processing takes too long. The client needs AI to speed it up and reduce manual effort.”
2. Framing B — “A credit officer spends six hours per case. Four of those are spent finding figures in documents and checking them. Cases wait nine days in queue. The bank is paying senior judgement rates for clerical work.”
3. Which one would you put in front of a CFO? Which one would you put in front of an engineer? Does your answer change?

PART 1

# Five framing traps to avoid tonight

| | |
|---|---|
| **Solution smuggling** | The technology appears in box 1 or 2. If “AI” is in your problem description, you have skipped the problem. |
| **The invisible baseline** | You write a target with no starting number. Then in week ten nobody can prove anything improved. |
| **The absent human** | The canvas describes a process, not a person. Ask: whose Monday morning gets better? |
| **Polite scope** | Box 8 is left vague to avoid an awkward conversation. That conversation only gets more expensive. |
| **Borrowed constraints** | You copy the compliance rules from another client. Ask this client. Their rules will differ. |

**QUICK CHECK** Read your canvas out loud. If it sounds like a proposal, rewrite it. It should sound like a complaint.

PART 2

# User stories

One sentence each. Who wants what, and why.



PART 2

# What a user story is — and what it is not

**As a [role], I want [something], so that [benefit].**

## It is

- One sentence, in the client's words
- A reminder to have a conversation
- Small enough to build and test
- Written from outside the system

## It is not

- A specification
- A description of screens or buttons
- A place to name technology
- Something the client never said

## The test

- Can you name the person?
- Can you say why they want it?
- Could you demo it in one minute?
- Would the client recognise it?

**WHY WE BOTHER** Six weeks from now somebody will argue about what to build. The story is the record of what the client asked for, in their language.

PART 2

# Your three personas — and what each one is really worried about

## Process owner

*Runs the operation day to day*

Throughput, queue length, staff time, and whether the team will actually use it

***"Will my Monday queue be shorter?"***

## Risk manager

*Owns what happens when it goes wrong*

Wrong decisions, exposure, how often people override the system

***"What happens when it is confidently wrong?"***

## Compliance auditor

*Has to prove it to somebody outside*

Evidence, traceability, retention, whether records can be changed

***"Show me the record for this decision from March."***

**THE ONE PEOPLE FORGET** The compliance auditor is not in the room when the project starts, and is the only one who can stop it going live. Two of your eight stories belong to that person. Write them first, not last — it is much cheaper to design for evidence than to add it in month four.

PART 2

# Weak story, strong story

**WEAK**

*As a user, I want the system to be fast, so that I am happy.*

→

**STRONG**

As a credit officer, I want each extracted figure to show the page it came from, so that I can check it without opening the full file.

**WEAK**

*As a manager, I want a dashboard with all the data.*

→

**STRONG**

As a process owner, I want to see how many cases are waiting on a human, so that I can move staff before the queue builds.

**WEAK**

*As compliance, I want the system to be GDPR compliant.*

→

**STRONG**

As a compliance auditor, I want to retrieve the full decision record for any case by reference, so that I can answer a regulator without contacting IT.

**WHAT CHANGED** The strong versions name a real person, describe something you could demo, and say why it matters. None of them mention technology.

PART 2

# Acceptance criteria — how you know a story is done

## Three words, in order

**GIVEN** the situation you start in

**WHEN** the thing that happens

**THEN** what must be true afterwards

worked example

```
GIVEN   a case file where the covenant
        schedule is missing

WHEN    the system runs extraction

THEN    the covenant field returns
        "not_found", and the case is
        flagged for human review
```

**WHY THIS MATTERS MORE THAN IT LOOKS** Notice what the example does. It describes a case where information is missing — and says the correct behaviour is to admit it. Most teams only write criteria for the happy path. The arguments in month three are always about the other paths.

**Write two criteria per story tonight:**

one for when everything works, and one for when something is missing, late, or wrong. The second one is where the real design decisions live.

PART 2 · SCENARIO

# Talk about this

DISCUSS · 6 MIN

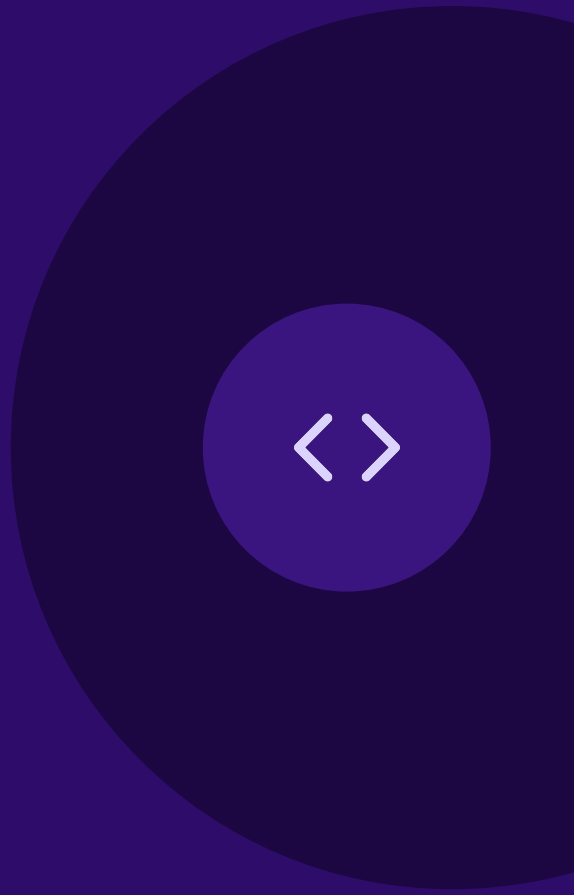
**You have written six user stories. The client's operations head is happy. Which two people have not been heard — and what will it cost you to find out in month four?**

1. All six stories came from the process owner, because she was the one who attended the workshop.
2. The risk manager was on leave. The compliance auditor was not invited — nobody thought to.
3. In your own accounts: who is the person who was not in the discovery room, and turned up later to block it?

PART 3

# The API contract

A written promise between two systems. You can read it without being a programmer.



PART 3

# What an API contract is, in plain words

> *"If you send me this, I promise to send you back that — and here is what happens when things go wrong."*

### It is a document, not code

OpenAPI is just a structured file. You can read it, review it and argue with it without writing a single line of software.

### Two teams build against it at once

The case system team can start work before DecisionStream exists, because the promise is already written down.

### It is what security review reads

Every field, and what kind of data it holds. Reviewers read the file instead of interviewing your team for two weeks.

### It is what stops surprises

If a field is not in the contract, it does not exist. No helpful extras that someone quietly starts depending on.

PART 3

# POST /case-application — reading it out loud

**POST** `/case-application` means *"Here is a new case. Please assess it."*

| Word | What it means | Why this one |
|---|---|---|
| **POST** | Create something new on the other system | Each case assessment is a new thing, not a lookup |
| **/case-application** | The name of the thing being created | Plain business language — not /api/v1/proc |
| **Request** | What you send: the case reference and documents | References, not files. Keeps personal data out of the call. |
| **Response** | What comes back, and how quickly | A ticket, not an answer. The work takes minutes. |
| **Status code** | A number saying what happened | 202 means accepted. 422 means the case cannot be assessed. |

**NOTICE THE NAMING** The endpoint is named after a business event that a client would recognise, not after a piece of technology. When the architecture changes in two years, this name still makes sense. That is not a style preference — it is how contracts survive.

PART 3

# Six questions that make a contract — no code needed

1. **What do you send?**
   Case reference, document references, decision date. Agree the smallest set that works.

2. **What comes back, and when?**
   Straight away, or later? A 340-page file cannot be read while somebody waits.

3. **What if the same thing is sent twice?**
   A network timeout must not create two assessments and two audit records.

4. **What happens when it goes wrong?**
   And can the caller try again? Say so clearly, or they will retry forever.

5. **Who is allowed to call it?**
   A valid login proves who is asking. It does not prove they may see this customer.

6. **What must NEVER come back?**
   A decision. A case status. Any sentence meant for a customer. This one is legal, not technical.

**QUESTION 6 IS THE ONE THAT MATTERS HERE** If your response carries a field called “outcome”, somebody downstream will write it straight into the case. That is an Article 22 problem, and it is now yours.

PART 3

# What that looks like written down

openapi.yaml — you will start from this file

```
paths:
  /case-application:
    post:
      summary: Submit a case for assessment
      parameters:
        - name: Idempotency-Key
          in: header
          required: true
      responses:
        '202':
          description: Accepted for processing
          headers:
            Location:
              description: Come back here for the result
        '409':
          description: Sent twice with different content
        '422':
          description: Case cannot be assessed right now
```

### It reads top to bottom

Path, then method, then what you send, then what comes back. That is the whole shape.

### “Idempotency-Key”

A label the caller puts on the request. If the same label arrives twice, we know it is a retry — not a second case.

### 202, not 200

200 means “here is your answer”. 202 means “I have accepted this and I am working on it”.

### 409 and 422 are not faults

They are answers. The caller learns something and can act on it.

### Your job tonight

Get the shape right. The detail comes in Sprint 1. Version 0.1 only has to be agreed, not finished.

PART 3 · SCENARIO

# Talk about this

DISCUSS · 6 MIN

**Two people in your pod disagree about the contract. One wants the decision returned immediately. The other wants a ticket number and a draft later. Who is right?**

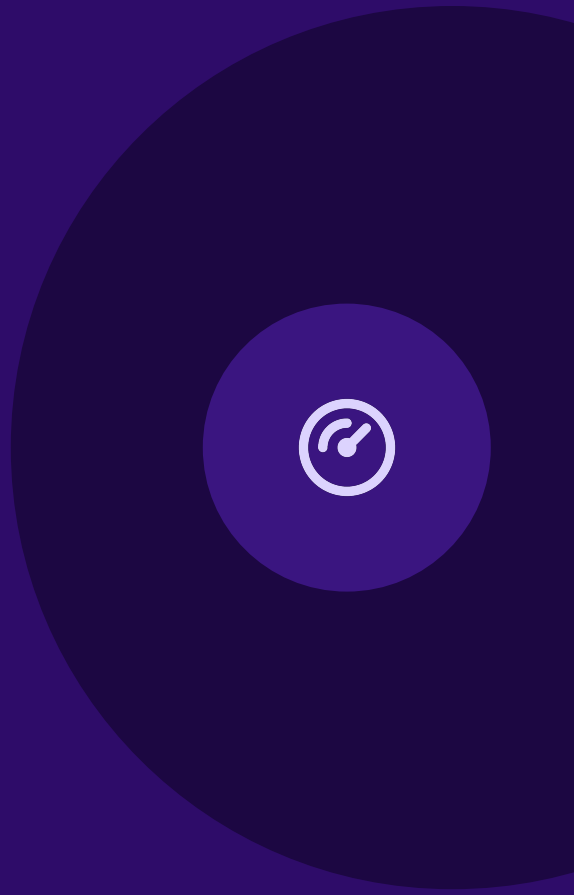
1. “The case system needs an answer on the screen. Make it return the decision in the response. One call, done.”
2. “It takes four minutes to read the file. Return a ticket, then send the draft when it is ready.”
3. There is a technical reason and a legal reason to prefer one of these. Name both.

**TRAINER** Legal reason first if they only find the technical one: a response field called “decision” invites a downstream system to write it into the case without a human. That is Article 22. The technical reason is simply that nothing should hold an HTTP connection open for four minutes.

PART 4

# Success criteria & NFRs

What “done” means — and what you are promising when you say it.



PART 4

# What “done” means for a proof of concept

## A POC proves one thing is possible

- It does not have to be production-ready.
- It does not have to handle every case.
- It does have to prove the thing everyone is unsure about.
- Decide tonight what that one thing is.

## Three tests of a success criterion

- Can you measure it?
- Can you fail it?
- Did someone with authority agree to it before you started?

**WEAK**

*“The system works and users find it helpful.”*

Nobody can fail this. So nobody can pass it either.

**STRONG**

“For 50 real anonymised cases, a structured decision profile returns within the agreed time, every extracted figure shows its source page, and an officer accepts the draft in at least 70% of them.”

**SET THE NUMBER TONIGHT, NOT LATER** Whatever percentage you choose, choose it now and write down who agreed. A target agreed after you have seen the results is not a target.

PART 4

# Non-functional requirements — the four that matter tonight

**Functional = what the system does. Non-functional = how well it must do it. A car moves. How fast, how safely, how quietly — those are non-functional.**

## Latency

*How long a person waits for an answer.*

We say P95, not average. P95 means: run 100 cases, 95 of them finish inside this time. Average hides the slow ones.

## SLA

*A promise with a consequence attached.*

99.9% uptime is about 43 minutes down per month. 99.5% is about 3.6 hours. The gap is a design decision and a price.

## Audit

*What gets recorded, for how long, and can it be changed.*

“Write once, never edited” is a storage choice made at design time. It cannot be added afterwards.

## Explainability

*Being able to say why the system gave this answer.*

In language a customer and a regulator both accept. Citations are how you get this — which is why they are in the design.

**REMEMBER** Every one of these numbers is a commercial promise. Sales signs them. Delivery lives with them.

PART 4 · SCENARIO

# Talk about this

DISCUSS · 7 MIN

**Sales has promised a two-second response. Your document extraction takes forty seconds on a 300-page file. The contract is signed on Friday. What do you do?**

1. Option A — go back and renegotiate the number. Who has that conversation, and what does it cost the relationship?
2. Option B — redefine what the two seconds applies to. Is that honest, or is it wordplay?
3. Option C — change the design so nobody waits. What does the client's screen look like then?

The real lesson is that the number should have been in the framing canvas before it reached a proposal — this scenario exists because framing happened after selling.

THE WORK · 45 MINUTES

# Three lanes, one use case, your own industry

## BUILD

*Data · AI · hands-on architects*

- Draft OpenAPI v0.1 for POST /case-application
- Paths, status codes, idempotency header
- Shape agreed, detail can wait for Sprint 1

## SHAPE

*Architects · Pod Leads · Account Mgmt*

- Complete the Problem Framing Canvas, all eight boxes
- Write the eight user stories, two per persona
- Two acceptance criteria per story

## GOVERN

*Sales · Account Mgmt · Business Analysts*

- Success criteria — measurable, failable, agreed
- The four NFRs with real numbers against them
- Define what “done” means for this POC

**FIRST FIVE MINUTES — DO THIS BEFORE ANYTHING ELSE** As a pod, choose your industry: credit underwriting, insurance claims, citizen service requests, healthcare prior-authorisation, or internal HR and vendor cases. Everything you write tonight is for that industry.

Tools: Problem Framing Canvas · Miro · user story template · OpenAPI editor · GitHub for the spec file

THE WORK

# What good looks like when you present

## Solid — expected at your level

- All eight canvas boxes filled
- Eight user stories in the right format
- An OpenAPI file that opens without errors
- Success criteria with numbers in them
- The four NFRs named and given values

## Senior — what we are watching for

- Box 4 names a real reason the last attempt failed
- Box 8 contains something the client will not like hearing
- Two stories belong to the compliance auditor
- One acceptance criterion covers missing information
- An NFR number that somebody has actually agreed to
- You can say which parts you are guessing

**HONEST BEATS COMPLETE** A canvas with two boxes marked “we do not know yet — we will ask on Thursday” scores higher than eight boxes of confident guesses.

THE DEFENCE · 20 MINUTES

# Questions each lane should expect

| | | |
|---|---|---|
| 1 | **Read me your problem statement.** | *Does it name a solution anywhere? If it does, rewrite it now.* |
| 2 | **Who suffers, and how do you know?** | *Did somebody tell you, or did you assume it?* |
| 3 | **Why has nobody fixed this already?** | *If you cannot answer box 4, you will meet the reason in month three.* |
| 4 | **Which of your eight stories would compliance reject?** | *If none, you have not written the auditor's stories properly.* |
| 5 | **Your NFR says P95 under five seconds.** | *Who agreed to that number, and what happens contractually if you miss it?* |
| 6 | **What is in box 8 that the client will argue about?** | *Scope you have not agreed is scope you have accepted.* |

COMMERCIAL CLOSE · 10 MINUTES

# Whose account does this fit?

You now have a framing canvas for a real industry. Name the account out loud. We will build the rest of this programme against real accounts, not a made-up one.

01 A problem statement you cannot fail is a wish, not a problem.

02 Box 4 tells you what will block you. Box 8 tells the client what you are not doing.

03 A user story names a person and says why. It never names technology.

04 The API contract is a document. If a field is not in it, it does not exist.

05 Every NFR number is a commercial promise. Agree it before you write it down.

*Homework: read your canvas to somebody who was not here. If they cannot tell you what the client should stop doing on Monday, fix it.*

BEFORE YOU LEAVE

**Canvas complete**
**OpenAPI v0.1 in GitHub**
**Industry chosen**

Tomorrow we turn this into an architecture.